

Deep Learning.

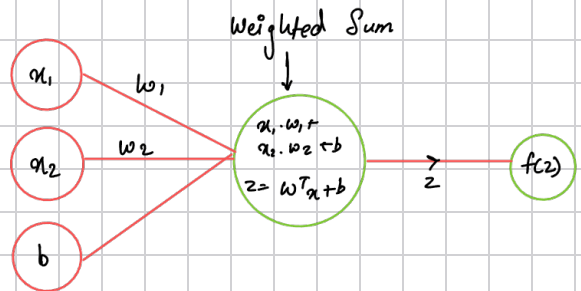
→ To solve more complex data and make processing more faster.

→ Machine learning was able to perform task very goodly but keep failing for data like images, audio.

→ Then comes Frank Rosenblatt and invented perceptron in 1970.

→ Perceptron is just like biological neurons.

- Having dendrites.
- Neuron
- Axon



→ But there is limitation of perceptron

Terminologies

① x



All of your input features.

② w



Weight according to importance of feature

③ Activation function



A function which will activate neuron to perform task.

{ in perceptron we use step function which is only good for binary classification }

Limitation with perceptron

↳ Now, as we know that perceptrons uses activating function as step function which are only good for Binary classification problem OR

the problems which are linearly separable.

⇒ The limitation of perceptron is NON-LINEAR SEPARABLE DATA

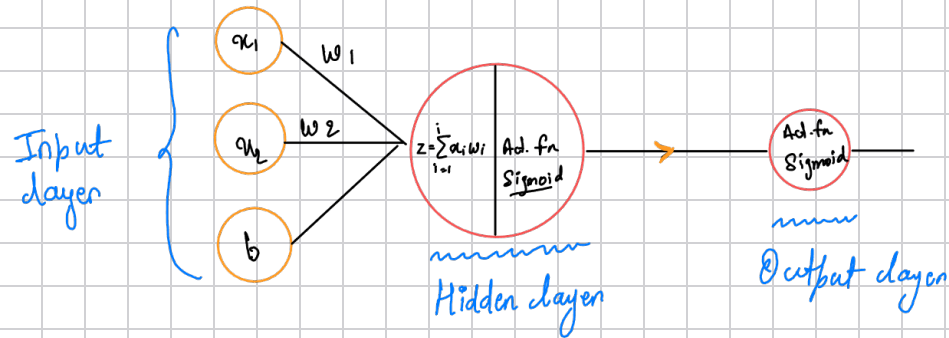
→ Perceptrons fails on non linear separable data and after which researchers come up with a new concept of multi layer perceptron

→ Multi layer perceptron makes multiple layers and those multiple layers will help to simplify and to process even more faster than before.

Now the question is how can we solve this problem?

↳ The answer is simply just change the activation function and add some more layers.

MULTI-LAYER PERCEPTRON



Activation function ↳ Sigmoid

$$\sigma = \frac{1}{1 + e^{-z}}$$

Always remember one thing
Sigmoid will never give us
the exact value 0 or 1.

Instead it will give us
the value between 0 to 1.

So, it can be 0.45, 0.75,
0.85, 0.25,

Hidden layer.

• Region - 1.



Weighted Sum

$$z = x_1 w_1 + x_2 w_2 + b$$

$$z = \sum x_i w_i + b$$

$$z = w^T x + b$$

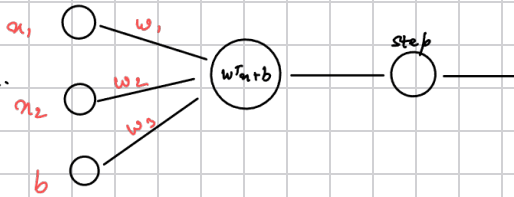
• Region - 2



Activation function

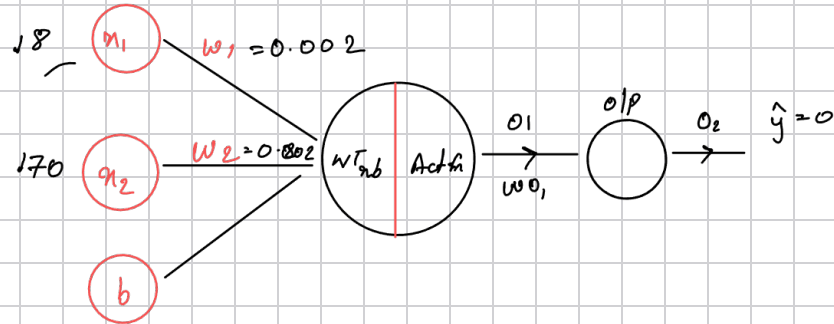
$$\sigma = \frac{1}{1 + e^{-z}}$$

Perceptron.



Hours (x_1) Attendance (x_2) Result (y)

8	70	1
7	50	0
6	40	1
7	30	1



Remember one thing that loss should be near to 0.

$$\begin{aligned} \text{loss function} &= |\hat{y} - y| \\ &= |0 - 1| \\ &= 1 \end{aligned}$$

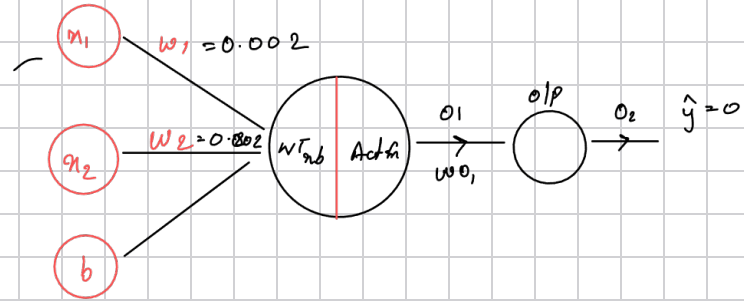
Forward Propagation.

1. Input layer with weight & bias.
2. Weighted Sum & activation function
3. Output layer
4. loss function (near to 0).

Backward Propagation.

1. Optimizers
2. weight updation
3. Vanishing gradient problem
4. loss calculation

Forward Propagation.



Backward Propagation.

1 epoch = forward propagation
+
Backward propagation